MASooda: Adversarial Threat Model

Owner: P5 (adversarial model and W-MSR resilience) · Status: **Draft v0.1 for team review** · Week 1 deliverable

Values marked *proposed* are P5's suggestions and need team agreement. Open questions are collected in Section 9.

1. Purpose and scope

This document fixes the adversary that the resilience layer is designed against and evaluated against. The core scenario matches the project plan: **one agent falsifies the position it broadcasts, starting at t = 60 s**, in a 60-agent, three-corridor simulation. Honest agents defend with W-MSR filtering. The same attack is run against the hierarchical, centralized and decentralized-undefended architectures under identical conditions (experiment E4). Everything not listed here is out of scope (Section 8).

The model follows the structure used in the resilient-consensus literature the team has read so far: who the adversary is, how many there are, what it can send, what it knows, and which parts of the system are trusted. Claims that depend on those papers are marked with [1] and [2] and listed in the references.

2. What is being attacked

Each tick, every agent broadcasts its state to neighbours in the communication graph (radius 40 m or k = 6). The adversary only tampers with this broadcast. The table shows who consumes broadcast data and who does not.

Module	Reads	Under attack?
P3 local behaviour (boids)	Neighbour positions and velocities for separation, alignment, cohesion	Yes. This is the main entry point of the attack.
P2 ADMM conflict resolution	Positions of agents at a merge point	Yes, if built from reported positions (see D2).
P4 overseer	Coarse snapshot (corridor occupancy)	No, treated as a trusted channel in core scope (D5).
P6 STL monitor and metrics	Ground-truth trajectories from P1	No. Safety is always judged on **true** positions, never on falsified ones.

3. Adversary model

3.1 Core specification

Attribute	Specification
Number of adversaries	One in the core scenario. The defence is designed for an upper bound F = 1; the actual number is unknown to honest agents.
Choice of attacker	Fixed before the run, from config (`malicious_ids`) or a seeded draw. Identical across architectures in E4 (*proposed*, D1).
Timing	Honest until t_a = 60 s (the base-config attack start time), then falsifies until the end of the run (180 s).
Attack type	Falsified **position** in the broadcast. No other field is falsified.
Physical behaviour	Honest. The attacker's true dynamics and local controller are unchanged, so it still moves as commanded; only its communication lies. This matches the non-cooperative-robot assumption in [2].
Collusion	None (single attacker), so the non-colluding case of [1] applies.

Attribute	Specification
Knowledge	The implemented attack is open-loop and non-adaptive: it does not need the topology or other agents' states. The W-MSR guarantee itself holds for arbitrary false values, so it does not depend on what the attacker knows. [1] and [2] do not specify attacker knowledge, so this is a P5 modelling choice.

3.2 Delivery modes (same attack type, two ways of sending it)

Both modes falsify position only. They differ in what each neighbour receives, using the two fault models in [1]:

- **Common mode (malicious in the sense of [1]).** The attacker sends the *same* false position to every neighbour. This is the core MVP scenario.
- **Per-neighbour mode (Byzantine in the sense of [1]).** The attacker sends a *different* false position to each neighbour. Run as an additional E4 variant, switched by a config flag.

3.3 Fault-count models and why a single attacker is easy to cover

The literature bounds misbehaving nodes three ways: **F-total** (at most F in the whole network), **F-local** (at most F in any node's neighbourhood) and **f-fraction local** (at most a fraction f of any node's neighbours). With exactly one attacker, the F-total bound with F = 1 holds, and F-local with F = 1 follows from it, since no neighbourhood can contain more than one attacker. So the adversary-count assumption of W-MSR is satisfied by construction, and the open question is the **graph-robustness condition** (Section 6).

3.4 Adversary objectives

- Reduce true minimum separation so STL robustness becomes negative (safety violations).
- Disturb formation and corridor merging so that throughput drops.
- Lengthen recovery time after a violation.

These map directly onto the E4 metrics: safety-violation count, throughput, recovery time and STL robustness.

3.5 What the adversary cannot do

- Change other agents' states or dynamics, or its own physical motion.
- Alter physically sensed range measurements.
- Forge another agent's identity (no Sybil attack) or send under a different id.
- Jam, delay, drop or replay messages.
- Falsify any field other than its broadcast position.
- Attack the overseer or its link. Overseer link failure is a separate experiment (E6), not an attack.

4. Trust assumptions

Item	Trusted?	Notes
Agent identifiers	Yes	Authentic and unforgeable. Authentication is not addressed in [2], so this is an explicit assumption that rules out Sybil attacks.
Own state of each honest agent	Yes	No self-spoofing or victim-side GPS attack.
Locally sensed range to a neighbour	Yes	Physically measured, so the attacker cannot change it. Ideal in core runs; optional bounded noise ε_r from config. Used only in the W-MSR fallback (Section 6.3).
Honest agents execute the protocol	Yes	Required by the guarantees in [1] and [2].
Honest agents know the bound F	Yes	They do not know which agent is malicious or how many there are.

Item	Trusted?	Notes
Reported neighbour broadcasts	**No**	This is the attack surface.
Overseer directive when link is up	Yes	Link failure is modelled separately (E6).
Delivery of messages	Yes	Reliable, synchronous per tick. No delay or loss in core scope.
Ground-truth states for STL and metrics	Yes	Experimenter's view from P1's trajectory log.

5. Attack specification

5.1 Definition

Let $p_m(t)$ be the true position of the malicious agent m and $q_m(t)$ the position it reports. Let Δ be the offset magnitude and u a unit vector. For $t < t_a$, $q_m(t) = p_m(t)$.

Core mode: constant offset (common delivery), for $t \geq t_a$:

```
q_m(t) = p_m(t) + Δ·u
```

Optional mode: ramp (a stealthier variant that grows slowly with rate ρ):

```
q_m(t) = p_m(t) + min(Δ, ρ·(t − t_a))·u
```

Per-neighbour delivery. Neighbour j receives its own offset direction u_j, drawn from a seeded generator keyed by (seed, receiver id), so runs are reproducible:

```
q_(m→j)(t) = p_m(t) + Δ·u_j
```

5.2 Configuration (`configs/base_config.yaml`, section `attack:`)

Key	Meaning	Default
enabled	Attack on or off. Off gives the no-op stub.	true in E4
malicious_ids	List of attacker ids. Empty means a seeded draw.	one id (D1)
start_time	Attack start t_a (shared with the base-config attack start time)	60 s
mode	`constant` or `ramp`	constant
delivery	`common` or `per_neighbor`	common
offset_magnitude Δ	Size of the falsification	25 m, *proposed*; sweep 5 / 10 / 25 / 50 m
ramp_rate ρ	Growth rate in ramp mode	1 m/s, *proposed*
direction_deg	Direction of u. Default derived from the base seed.	seeded

The default Δ = 25 m is a first guess relative to d_{min} = 8 m and the 40 m communication radius. It should be tuned in Week 2 using the plain-averaging baseline, so that the undefended system visibly breaks.

5.3 Interface behaviour

```
inject_attack(agent_states, attack_config, t) -> AttackState
```

- Returns an **empty** `malicious` list and empty `falsified` dict when the attack is disabled or $t < t_a$. The Week 1 stub always does this.
- Otherwise `malicious` holds the attacker id and `falsified` holds the false positions: sender id to position in common mode, sender id to a receiver-to-position map in per-neighbour mode. This needs agreement before the schema freeze (D3).
- It is a pure function: it is deterministic for a given seed and never mutates `agent_states`.
- **Message layer.** P1 builds the reported neighbour states by overlaying `falsified` on the true states (proposed, D4). P3 and P2 read the reported values, or the W-MSR-filtered values. STL and

metrics read the true states.

6. Defence assumptions and guarantee conditions

6.1 W-MSR as used in MASooda

Each honest agent runs W-MSR with parameter f = F = 1 on a scalar quantity, applying it to the x and y coordinates independently. It sorts the received values, discards up to f values above its own and up to f below, and averages the rest. Per the interface, `wmsr_filter` returns a filtered estimate of the neighbour information used by the local behaviour (for example the neighbour centroid for cohesion and mean velocity for alignment).

6.2 What W-MSR guarantees, and under which conditions

Property	Condition (from notes on [1] and [2])	For F = 1
Safety: honest values stay within the range of the honest agents' initial values	Holds for W-MSR whenever the number of misbehaving neighbours is within the filtering parameter. No graph condition needed.	At most 1 attacker per neighbourhood (true by construction)
Agreement: honest agents converge to a common value	Graph is (2F + 1)-robust under the F-total, F-local and Byzantine models.	3-robust
Easier-to-check sufficient condition	Algebraic connectivity $\lambda_2 > 4F$ (a sufficient, not necessary, condition used in [2])	$\lambda_2 > 4$
Adversary bound	Number of non-cooperative agents ≤ F, with F known to honest agents	1

6.3 Where the guarantee can fail in MASooda, and the plan

- **Spatial graphs are not built to be robust.** A radius or k-nearest-neighbour graph over moving agents can fail (2F + 1)-robustness, and exact robustness checking is expensive. *Plan:* as an experimenter-side diagnostic (not used by the agents), log λ_2 of the true communication graph every monitoring period and report the fraction of time $\lambda_2 > 4F$ in E4. Where the condition fails, claims about the defence are empirical only.
- **Low degree.** With fewer than 2F + 1 = 3 neighbours, W-MSR cannot filter meaningfully. *Plan:* treat this like the interface's failure case below.
- **Fallback.** If more than 2f reports look like outliers, or degree is below 2F + 1, the interface falls back to a trusted-subset heuristic: keep only neighbours whose reported position is consistent with the sensed range, $|\|q_j - p_i\| - r_{ij}| \le \tau$. This check is used **only** in the fallback, so that E4 tests W-MSR itself rather than a range-consistency detector. Because the attacker cannot alter sensed range, a large Δ is easy to catch by this check, which is why the Δ sweep and the ramp mode matter.
- **Exceeding the bound.** If the real number of adversaries exceeds F, no guarantee holds. [2] also notes that resilience of the consensus rule does not carry over to any mechanism that computes connectivity from reported data; MASooda has no connectivity controller, so λ_2 is a diagnostic only.

7. Evaluation hooks for E4

The metrics are those defined in the project plan: throughput, safety-violation count (ticks with ρ < 0), recovery time and STL robustness. P5 adds the following observables, which also serve the failure-mode matrix (case 4, malicious neighbours filtered):

- Number of reports discarded by W-MSR per tick, and how many came from the attacker.
- Number of fallback activations, and the fraction of time $\lambda_2 > 4F$.
- Deviation of the filtered estimate from the estimate computed on true positions.

Planned runs: with and without the attack; f = 0 (plain averaging) versus f = 1; common versus per-neighbour delivery; Δ sweep; constant versus ramp mode; several seeds. Across architectures the seed, agent population, initial conditions, attacker id and attack parameters are identical.

8. Out of scope and known limitations

- More than one attacker, collusion, and Sybil identities.
- Falsifying velocity, priority or destination. Priority is owned and assigned by the overseer (P4) in the shared schema, so an agent cannot self-declare it in this design. The project summary mentions position *or* intent; intent (destination or path) falsification is documented here as an optional extension only.
- Jamming, denial of service, replay, and message delay.
- Spoofing of an honest agent's own sensors, including its range sensing.
- Attacks on the overseer or its link.
- An attacker that adapts to W-MSR or optimises against it.
- Formal safety proofs for ADMM or separation under attack. Safety is measured empirically through STL robustness.

9. Open decisions for the team

ID	Question	With	P5 proposal
D1	Which agent is malicious, and of which type?	P1	Delivery agent chosen by the seeded draw; fixed across architectures. Vary the type later.
D2	What does P3 use for separation and cohesion: reported positions, filtered values, or sensed relative range? What exactly does W-MSR filter?	P3	Undefended baseline uses reported values; hierarchical uses W-MSR-filtered aggregates. Needs agreement because it decides where the attack bites.
D3	Structure of `AttackState.falsified` (schema freezes end of Week 2)	P1	Sender-keyed map, with a receiver-keyed inner map in per-neighbour mode.
D4	Who overlays falsified data onto neighbour states?	P1	P1's message layer, so P3 stays unaware of the attack.
D5	Is the overseer snapshot built from true states?	P4	Yes, trusted in core scope.
D6	Default Δ and ρ	All	Tune in Week 2 against plain averaging.
D7	Is the attacker counted in throughput and violation metrics?	P6	Yes, because it moves honestly.

10. Plan and next steps for P5

Week	P5 work on this model
1	This document; no-op `inject_attack` with the final signature; agree D1 to D5 with the owners.
2	Real `inject_attack` (constant, ramp, per-neighbour); plain-averaging baseline showing corruption by one liar; tune Δ.
3	W-MSR filter; W-MSR versus plain averaging on injected false reports; λ_2 diagnostic.
4	Wire attack and W-MSR into the main loop with P1; one attacker, fixed start time.
5	Decentralized-undefended architecture; start E4 under identical conditions.
6	Final resilience plots over several seeds.

References

[1] Paper 1: the W-MSR and robust-networks paper (notes on fault models, network robustness, F-total, F-local and f-fraction models). *Add full citation.*

[2] Paper 2: resilient flocking with algebraic-connectivity control (notes on the non-cooperative-robot model, (2F + 1)-robustness and $\lambda_2 > 4F$). *Add full citation.*

[3] MASooda six-week project plan (workflow.pdf): schema, interfaces, configuration, experiment matrix E1 to E6.